

PyramidEdu

Notes Upload + RAG + Gemini AI Chatbot Complete Implementation Guide

■ PDF Upload Cloudinary Storage	■ Text Extraction pdf-parse / PyPDF
>■ Chunking 500–1000 token chunks	■ Embeddings Gemini text-embedding-004
■■ Vector DB pgvector on Neon PostgreSQL	■ RAG Chatbot Gemini 1.5 Flash

This guide is a complete, step-by-step implementation manual for the PyramidEdu Notes Upload + RAG (Retrieval Augmented Generation) system. It covers every phase — from Cloudinary setup to deploying a Gemini-powered chatbot that answers only from teacher-uploaded study materials.

Generated: June 14, 2026 · Stack: Node.js / TypeScript · Next.js · PostgreSQL (Neon) · Cloudinary · Google Gemini API

Table of Contents

1		
.	System Architecture Overview	3
2		
.	Prerequisites & Environment Setup	3
3		
.	Neon Database Setup (pgvector)	4
4		
.	Prisma Schema — Notes & Chunks	5
5		
.	Cloudinary Setup & PDF Upload	6
6		
.	PDF Text Extraction (Backend)	8
7		
.	Text Chunking Pipeline	9
8		
.	Embedding Generation (Gemini)	10
9		
.	Storing Embeddings in pgvector	11
10		
.	Next.js API Routes — Upload Endpoint	12
11		
.	Teacher Upload UI Component	13
12		
.	RAG Chatbot — Query Pipeline	14
13		
.	Chatbot API Route	15
14		
.	Student Chatbot UI	16
15		
.	End-to-End Flow Diagram	17
16		
.	Testing Checklist	17

1

7

. Troubleshooting & Tips

1

8

1. System Architecture Overview

PyramidEdu's RAG system follows a two-phase architecture: an **ingestion pipeline** (run once per uploaded PDF) and a **query pipeline** (run on every student question). Both phases communicate through a shared **vector store** in Neon PostgreSQL.

INGESTION PIPELINE (Teacher → DB)	QUERY PIPELINE (Student → Answer)
1. Teacher submits PDF via Next.js UI 2. Multer/Cloudinary middleware uploads file 3. Cloudinary returns secure URL 4. Backend fetches PDF from URL 5. pdf-parse extracts raw text 6. Text splitter creates 500-token chunks 7. Each chunk → Gemini embedding API 8. Vector stored in note_chunks (pgvector)	1. Student types question in chatbot 2. Question → Gemini embedding API 3. pgvector cosine similarity search 4. Top-5 relevant chunks retrieved 5. Context + question → Gemini LLM 6. Gemini generates grounded answer 7. Answer displayed to student 8. "Not found" if no relevant context

2. Prerequisites & Environment Setup

2.1 Required Accounts

- Neon Database account — neon.tech (free tier works)
- Cloudinary account — cloudinary.com (free tier: 25GB storage)
- Google AI Studio account — aistudio.google.com (get Gemini API key)
- Node.js 18+ and npm/yarn installed locally

2.2 NPM Packages to Install

Package	Purpose	Install Command
@prisma/client	ORM client for PostgreSQL	npm install @prisma/client
prisma	Prisma CLI (dev dependency)	npm install -D prisma
cloudinary	Cloudinary Node.js SDK	npm install cloudinary
multer	Multipart form-data parser	npm install multer
multer-storage-cloudinary	Multer + Cloudinary storage	npm install multer-storage-cloudinary
pdf-parse	Extract text from PDFs	npm install pdf-parse
@google/generative-ai	Official Gemini SDK	npm install @google/generative-ai
axios	HTTP client (fetch PDF from URL)	npm install axios

Package	Purpose	Install Command
@types/multer	TypeScript types for multer	npm install -D @types/multer
@types/pdf-parse	TypeScript types for pdf-parse	npm install -D @types/pdf-parse

2.3 Environment Variables (.env)

Variable	Value / Description	Where to Get It
DATABASE_URL	postgresql://user:pass@host/db?sslmode=require	Neon dashboard → Connection string
CLOUDINARY_CLOUD_NAME	your_cloud_name	Cloudinary dashboard → Settings
CLOUDINARY_API_KEY	123456789012345	Cloudinary dashboard → Settings
CLOUDINARY_API_SECRET	your_api_secret_here	Cloudinary dashboard → Settings
GEMINI_API_KEY	Alza...your_key_here	aistudio.google.com → Get API Key
NEXT_PUBLIC_APP_URL	http://localhost:3000	Your Next.js app base URL

■ **Never commit .env to Git. Add it to .gitignore immediately. Use Vercel Environment Variables for production deployment.**

3. Neon Database Setup (pgvector)

Neon supports the **pgvector** extension natively. You must enable it before running any Prisma migrations that use vector columns. Follow these exact steps:

1 Open Neon Console

Go to console.neon.tech → select your project → click on 'SQL Editor' in the left sidebar.

2 Enable pgvector extension

Paste and run the SQL command below. This only needs to be done once per database.

3 Verify installation

Run: `SELECT * FROM pg_extension WHERE extname = 'vector';` — you should see one row returned.

4 Set connection string

Copy the connection string from Neon dashboard and paste it into your `.env` as `DATABASE_URL`. Make sure to add `?sslmode=require` at the end if not already present.

3.1 SQL: Enable pgvector

```
-- Run this in Neon SQL Editor BEFORE prisma migrate
CREATE EXTENSION IF NOT EXISTS vector;

-- Verify it worked:
SELECT extname, extversion FROM pg_extension WHERE extname = 'vector';
-- Expected output: extname='vector', extversion='0.7.x'
```

3.2 Prisma Configuration for pgvector

Prisma does not natively support vector types yet. We use the **prisma-client-js** with raw SQL for vector operations. Here is the required `prisma/schema.prisma` configuration:

```
// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

datasource db {
  provider = "postgresql"
  url = env("DATABASE_URL")
  extensions = [vector]
}
```

■ After adding `previewFeatures = ["postgresqlExtensions"]`, run: `npx prisma generate` to regenerate the client.

4. Prisma Schema — Notes & Chunks Tables

Add the following models to your **schema.prisma**. The embedding field uses `Unsupported('vector(768)')` because Prisma does not yet have first-class vector support — this is correct and expected. Raw SQL is used for vector operations.

```
// Add to prisma/schema.prisma

model Note {
  id String @id @default(uuid())
  title String
  subject String
  batch String
  description String? @db.Text
  fileUrl String @map("file_url")
  uploadedBy String @map("uploaded_by") // teacher userId
  isActive Boolean @default(true) @map("is_active")
  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @updatedAt @map("updated_at")

  chunks NoteChunk[]

  @@index([subject])
  @@index([batch])
  @@map("notes")
}

model NoteChunk {
  id String @id @default(uuid())
  noteId String @map("note_id")
  chunkIndex Int @map("chunk_index")
  chunkText String @db.Text @map("chunk_text")
  // embedding stored as pgvector – 768 dims for Gemini text-embedding-004
  embedding Unsupported("vector(768)")?
  createdAt DateTime @default(now()) @map("created_at")

  note Note @relation(fields: [noteId], references: [id], onDelete: Cascade)

  @@index([noteId])
  @@map("note_chunks")
}
```

4.1 Migration Commands

```
# Step 1: Generate Prisma client
npx prisma generate

# Step 2: Create and apply the migration
npx prisma migrate dev --name add_notes_rag

# Step 3: Create the vector index (run in Neon SQL Editor)
CREATE INDEX ON note_chunks USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);

-- IVFFlat index dramatically speeds up similarity searches.
-- Use lists = sqrt(number_of_rows) as a rule of thumb.
```

```
-- For < 1000 rows, lists = 10 is fine.
```

■ The vector index (IVFFlat) must be created AFTER inserting at least some data, otherwise it will not work properly. Create the index after your first batch of embeddings.

5. Cloudinary Setup & PDF Upload

5.1 Cloudinary Configuration File

Create this file at **lib/cloudinary.ts** in your Next.js project:

```
// lib/cloudinary.ts
import { v2 as cloudinary } from 'cloudinary';

cloudinary.config({
  cloud_name : process.env.CLOUDINARY_CLOUD_NAME!,
  api_key : process.env.CLOUDINARY_API_KEY!,
  api_secret : process.env.CLOUDINARY_API_SECRET!,
  secure : true, // always use HTTPS
});

export default cloudinary;
```

5.2 Upload Utility Function

Create **lib/uploadPDF.ts** — this handles the actual upload to Cloudinary:

```
// lib/uploadPDF.ts
import cloudinary from './cloudinary';
import { Readable } from 'stream';

export interface UploadResult {
  secure_url : string;
  public_id : string;
  bytes : number;
}

export async function uploadPDFtoCloudinary(
  buffer : Buffer,
  fileName : string
): Promise<UploadResult> {
  return new Promise((resolve, reject) => {
    const stream = cloudinary.uploader.upload_stream(
      {
        folder : 'pyramidEdu/notes/pdfs',
        resource_type : 'raw', // required for PDFs
        public_id : `${Date.now()}_${fileName.replace(/\.\pdf$/i, '')}`,
        allowed_formats: ['pdf'],
        overwrite : false,
      },
      (error, result) => {
        if (error || !result) return reject(error);
        resolve({
          secure_url : result.secure_url,
          public_id : result.public_id,
          bytes : result.bytes,
        });
      }
    );
    // Convert Buffer → Readable stream
    const readable = new Readable();
```

```
readable.push(buffer);
readable.push(null);
readable.pipe(stream);
});
}
```

Why resource_type: 'raw'? Cloudinary treats PDFs as 'raw' files by default. Using 'image' or 'video' will fail.
Always use 'raw' for PDF uploads.

5.3 File Validation Middleware

Create **lib/validatePDF.ts** to validate before uploading:

```
// lib/validatePDF.ts

const MAX_SIZE_BYTES = 10 * 1024 * 1024; // 10 MB

export interface ValidationResult {
  valid : boolean;
  message : string;
}

export function validatePDFFile(
  file: { size: number; type: string; name: string }
): ValidationResult {
  if (file.type !== 'application/pdf') {
    return { valid: false, message: 'Only PDF files are allowed.' };
  }
  if (file.size > MAX_SIZE_BYTES) {
    return { valid: false, message: 'File size must be 10 MB or less.' };
  }
  if (!file.name.toLowerCase().endsWith('.pdf')) {
    return { valid: false, message: 'File must have a .pdf extension.' };
  }
  return { valid: true, message: 'Valid PDF file.' };
}
```

6. PDF Text Extraction (Backend)

After the PDF is uploaded to Cloudinary, download it using axios and extract text with **pdf-parse**. This text becomes the raw input for the chunking pipeline.

```
// lib/extractPDFText.ts
import axios from 'axios';
import pdfParse from 'pdf-parse';

export async function extractTextFromPDFUrl(
  cloudinaryUrl: string
): Promise<string> {
  // 1. Download PDF buffer from Cloudinary
  const response = await axios.get<ArrayBuffer>(cloudinaryUrl, {
    responseType: 'arraybuffer',
    timeout: 30_000, // 30s timeout
  });

  const buffer = Buffer.from(response.data);

  // 2. Extract text using pdf-parse
  const data = await pdfParse(buffer);

  // 3. Clean and normalize the extracted text
  let text = data.text
    .replace(/\\r\\n/g, '\\n') // normalize line endings
    .replace(/\\n{3,}/g, '\\n\\n') // collapse excessive blank lines
    .replace(/[ \\t]{2,}/g, ' ') // collapse multiple spaces
    .replace(/^[^\\x20-\\x7E\\n]/g, ' ') // remove non-ASCII control chars
    .trim();

  if (!text || text.length < 50) {
    throw new Error(
      'PDF text extraction failed or PDF is scanned (image-based). ' +
      'Please upload a text-based PDF.'
    );
  }

  console.log(`Extracted ${text.length} characters from PDF`);
  return text;
}
```

■ pdf-parse cannot extract text from scanned/image PDFs. If teachers need to upload scanned notes, consider adding Tesseract.js OCR as a fallback. For now, instruct teachers to only upload text-based (digital) PDFs.

7. Text Chunking Pipeline

Chunking splits long text into smaller, overlapping pieces so that each piece can be embedded independently. Overlap ensures that sentences near chunk boundaries are captured in at least one chunk's context.

Chunk Size: 800 words (~1000 tokens) · Overlap: 100 words · Why: Gemini text-embedding-004 handles up to 2048 tokens. 800 words gives a comfortable margin while keeping chunks semantically rich.

```
// lib/chunkText.ts

export interface TextChunk {
  index : number;
  text : string;
}

export function chunkText(
  text : string,
  chunkSize : number = 800, // words per chunk
  overlap : number = 100 // overlapping words
): TextChunk[] {
  // Split into individual words
  const words = text.split(/\s+/).filter(Boolean);
  const chunks : TextChunk[] = [];
  let index = 0;
  let start = 0;

  while (start < words.length) {
    const end = Math.min(start + chunkSize, words.length);
    const text = words.slice(start, end).join(' ');

    // Only keep chunks with meaningful content (> 20 words)
    if (text.trim().split(/\s+/).length > 20) {
      chunks.push({ index: index++, text: text.trim() });
    }

    // Advance by chunkSize - overlap to create sliding window
    start += chunkSize - overlap;
  }

  console.log(`Created ${chunks.length} chunks from ${words.length} words`);
  return chunks;
}
```

8. Embedding Generation (Gemini)

Each text chunk is converted to a 768-dimensional vector using **Gemini text-embedding-004**. This vector mathematically represents the semantic meaning of the chunk. Similar content produces similar vectors.

```
// lib/generateEmbedding.ts
import { GoogleGenerativeAI } from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);

// Gemini text-embedding-004 produces 768-dimensional vectors
const embeddingModel = genAI.getGenerativeModel({
  model: 'text-embedding-004',
});

export async function generateEmbedding(text: string): Promise<number[]> {
  const result = await embeddingModel.embedContent(text);
  return result.embedding.values; // number[] of length 768
}

// Batch embed multiple chunks with rate limiting
export async function generateEmbeddingsBatch(
  chunks: string[],
  delayMs: number = 200 // avoid API rate limits
): Promise<number[][]> {
  const embeddings: number[][] = [];

  for (let i = 0; i < chunks.length; i++) {
    const embedding = await generateEmbedding(chunks[i]);
    embeddings.push(embedding);

    // Rate limit: 200ms between requests
    if (i < chunks.length - 1) {
      await new Promise(r => setTimeout(r, delayMs));
    }

    // Log progress
    console.log(`Embedded chunk ${i + 1}/${chunks.length}`);
  }

  return embeddings;
}
```

Rate Limits: Gemini embedding API allows 1500 requests/minute on free tier. The 200ms delay adds safety margin. For large PDFs (50+ chunks), consider increasing delayMs to 500ms or using the batch embedding endpoint.

9. Storing Embeddings in pgvector

Prisma does not support vector column writes natively, so we use **prisma.\$executeRaw** with tagged template literals to safely insert embeddings into the `note_chunks` table.

```
// lib/storeEmbeddings.ts
import { prisma } from './prisma'; // your Prisma client singleton
import { TextChunk } from './chunkText';
import { Prisma } from '@prisma/client';

export async function storeNoteWithChunks(
  noteData: {
    title : string;
    subject : string;
    batch : string;
    description?: string;
    fileUrl : string;
    uploadedBy : string;
  },
  chunks : TextChunk[],
  embeddings : number[][]
): Promise<string> {
  // 1. Insert the parent note record
  const note = await prisma.note.create({
    data: {
      title : noteData.title,
      subject : noteData.subject,
      batch : noteData.batch,
      description: noteData.description,
      fileUrl : noteData.fileUrl,
      uploadedBy : noteData.uploadedBy,
    },
  });

  // 2. Insert each chunk with its embedding using raw SQL
  for (let i = 0; i < chunks.length; i++) {
    const vectorLiteral = `[${embeddings[i].join(',')}]`;

    await prisma.$executeRaw`
    INSERT INTO note_chunks (id, note_id, chunk_index, chunk_text, embedding, created_at)
    VALUES (
      gen_random_uuid(),
      ${note.id},
      ${chunks[i].index},
      ${chunks[i].text},
      ${vectorLiteral}::vector,
      NOW()
    )`;
  };
}

console.log(`Stored note ${note.id} with ${chunks.length} chunks`);
return note.id;
}
```

■ Create a Prisma singleton in lib/prisma.ts to avoid creating multiple Prisma connections in Next.js: export const prisma = globalThis.prisma ?? new PrismaClient(); if (process.env.NODE_ENV !== 'production') globalThis.prisma = prisma;

10. Next.js API Route — Upload Endpoint

Create **app/api/notes/upload/route.ts** (App Router). This single endpoint orchestrates the entire ingestion pipeline: validate → upload → extract → chunk → embed → store.

```
// app/api/notes/upload/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { uploadPDFToCloudinary } from '@lib/uploadPDF';
import { extractTextFromPDFUrl } from '@lib/extractPDFText';
import { chunkText } from '@lib/chunkText';
import { generateEmbeddingsBatch } from '@lib/generateEmbedding';
import { storeNoteWithChunks } from '@lib/storeEmbeddings';
import { validatePDFFile } from '@lib/validatePDF';

export async function POST(req: NextRequest) {
  try {
    // 1. Parse multipart form data
    const formData = await req.formData();
    const file = formData.get('file') as File;
    const title = formData.get('title') as string;
    const subject = formData.get('subject') as string;
    const batch = formData.get('batch') as string;
    const description = formData.get('description') as string | undefined;
    const uploadedBy = formData.get('teacherId') as string;

    // 2. Validate inputs
    if (!file || !title || !subject || !batch || !uploadedBy) {
      return NextResponse.json({ error: 'Missing required fields' }, { status: 400 });
    }
    const validation = validatePDFFile({ size: file.size, type: file.type, name: file.name });
    if (!validation.valid) {
      return NextResponse.json({ error: validation.message }, { status: 400 });
    }

    // 3. Upload to Cloudinary
    const buffer = Buffer.from(await file.arrayBuffer());
    const uploaded = await uploadPDFToCloudinary(buffer, file.name);

    // 4. Extract text from PDF
    const rawText = await extractTextFromPDFUrl(uploaded.secure_url);

    // 5. Chunk the text
    const chunks = chunkText(rawText, 800, 100);

    // 6. Generate embeddings for all chunks
    const embeddings = await generateEmbeddingsBatch(chunks.map(c => c.text));

    // 7. Store note + chunks in database
    const noteId = await storeNoteWithChunks(
      { title, subject, batch, description, fileUrl: uploaded.secure_url, uploadedBy },
      chunks,
      embeddings
    );

    return NextResponse.json({
      success: true,
      noteId,
    });
  } catch (error) {
    // Handle error
  }
}
```

```
chunksCreated: chunks.length,  
fileUrl: uploaded.secure_url,  
});  
  
} catch (err: any) {  
  console.error('Upload error:', err);  
  return NextResponse.json({ error: err.message }, { status: 500 });  
}  
}
```

11. Teacher Upload UI Component

Create **components/NoteUploadForm.tsx**. This component handles file selection, validation feedback, and upload progress display.

```
// components/NoteUploadForm.tsx
'use client';
import { useState, useRef } from 'react';

export default function NoteUploadForm({ teacherId }: { teacherId: string }) {
  const [title, setTitle] = useState('');
  const [subject, setSubject] = useState('');
  const [batch, setBatch] = useState('');
  const [desc, setDesc] = useState('');
  const [file, setFile] = useState<File | null>(null);
  const [status, setStatus] = useState<'idle' | 'uploading' | 'done' | 'error'>('idle');
  const [message, setMessage] = useState('');
  const fileRef = useRef<HTMLInputElement>(null);

  const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const f = e.target.files?.[0];
    if (!f) return;
    if (f.type !== 'application/pdf') {
      setMessage('Only PDF files allowed.');
```

```

setFile(null);
if (fileRef.current) fileRef.current.value = '';
} catch (err: any) {
setStatus('error');
setMessage(`Error: ${err.message}`);
}
};

return (
<form onSubmit={handleSubmit} className='space-y-4 max-w-lg'>
<input value={title} onChange={e=>setTitle(e.target.value)}
placeholder='Note Title *' required className='input-class' />
<input value={subject} onChange={e=>setSubject(e.target.value)}
placeholder='Subject *' required className='input-class' />
<input value={batch} onChange={e=>setBatch(e.target.value)}
placeholder='Batch / Class *' required className='input-class' />
<textarea value={desc} onChange={e=>setDesc(e.target.value)}
placeholder='Description (optional)' className='input-class' />
<input ref={fileRef} type='file' accept='application/pdf'
onChange={handleFileChange} required />
{file && <p className='text-sm text-gray-600'>Selected: {file.name}</p>}
{message && <p className={status==='error'? 'text-red-500': 'text-blue-600'}>{message}</p>}
<button type='submit' disabled={status==='uploading'}
className='btn-primary'>
{status === 'uploading' ? 'Processing...' : 'Upload Notes'}
</button>
</form>
);
}

```

12. RAG Chatbot — Query Pipeline

The query pipeline converts a student's question to a vector, searches pgvector for similar chunks, and passes them as context to Gemini to generate a grounded answer.

```
// lib/ragQuery.ts
import { prisma } from './prisma';
import { generateEmbedding } from './generateEmbedding';
import { GoogleGenerativeAI } from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);
const llm = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });

interface RelevantChunk {
  chunk_text : string;
  note_title : string;
  similarity : number;
}

// Step 1: Find top-K relevant chunks via cosine similarity
async function findRelevantChunks(
  questionEmbedding : number[],
  topK : number = 5
): Promise<RelevantChunk[]> {
  const vectorLiteral = `[${questionEmbedding.join(',')}]`;

  const results = await prisma.$queryRaw<RelevantChunk[]>`
  SELECT
  nc.chunk_text,
  n.title AS note_title,
  1 - (nc.embedding <=> ${vectorLiteral}::vector) AS similarity
  FROM note_chunks nc
  JOIN notes n ON n.id = nc.note_id
  WHERE n.is_active = true
  ORDER BY nc.embedding <=> ${vectorLiteral}::vector
  LIMIT ${topK}
  `;

  // Only return chunks with meaningful similarity (> 0.5)
  return results.filter(r => r.similarity > 0.5);
}

// Step 2: Build prompt and call Gemini LLM
export async function ragAnswer(
  question : string,
  subject ? : string // optional filter by subject
): Promise<string> {
  // Embed the question
  const qEmbedding = await generateEmbedding(question);

  // Find relevant chunks
  const chunks = await findRelevantChunks(qEmbedding, 5);

  // No relevant content found
  if (chunks.length === 0) {
    return 'Not available in study materials. Please refer to your teacher or textbook.';
  }
}
```

```
// Build context string
const context = chunks
.map((c, i) => `[Source: ${c.note_title}]\n${c.chunk_text}`)
.join('\n\n---\n\n');

// Construct RAG prompt
const prompt = `You are PyramidEdu's academic assistant.
Answer ONLY using the context below from teacher-uploaded notes.
If the answer is not in the context, say exactly:
'Not available in study materials.'
Do not add information beyond what is provided.

CONTEXT FROM NOTES:
${context}

STUDENT QUESTION: ${question}

ANSWER:`;

const result = await llm.generateContent(prompt);
const response = result.response;
return response.text();
}
```

13. Chatbot API Route

Create `app/api/chat/route.ts`:

```
// app/api/chat/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { ragAnswer } from '@lib/ragQuery';
import { prisma } from '@lib/prisma';

export async function POST(req: NextRequest) {
  try {
    const { question, userId, conversationId } = await req.json();

    if (!question?.trim()) {
      return NextResponse.json({ error: 'Question is required' }, { status: 400 });
    }

    // Get answer via RAG pipeline
    const answer = await ragAnswer(question);

    // Persist the conversation (optional but recommended)
    if (conversationId && userId) {
      await prisma.chatMessage.createMany({
        data: [
          { conversationId, role: 'user', content: question },
          { conversationId, role: 'assistant', content: answer },
        ],
      });
    }

    return NextResponse.json({ answer });
  } catch (err: any) {
    console.error('Chat error:', err);
    return NextResponse.json({ error: 'Failed to generate answer' }, { status: 500 });
  }
}
```

14. Student Chatbot UI Component

```
// components/RAGChatbot.tsx
'use client';
import { useState } from 'react';

interface Message { role: 'user' | 'assistant'; text: string; }

export default function RAGChatbot({ userId }: { userId: string }) {
  const [messages, setMessages] = useState<Message[]>([
    { role: 'assistant', text: 'Hi! Ask me anything about your study notes.' }
  ]);
  const [input, setInput] = useState('');
  const [loading, setLoading] = useState(false);

  const sendMessage = async () => {
    if (!input.trim() || loading) return;
    const question = input.trim();
```

```

setMessages(prev => [...prev, { role: 'user', text: question }]);
setInput('');
setLoading(true);

try {
  const res = await fetch('/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ question, userId }),
  });
  const data = await res.json();
  setMessages(prev => [...prev, { role: 'assistant', text: data.answer }]);
} catch {
  setMessages(prev => [...prev, { role: 'assistant',
    text: 'Sorry, something went wrong. Please try again.' }]);
} finally {
  setLoading(false);
}
};

return (
  <div className='flex flex-col h-[600px] border rounded-xl overflow-hidden'>
    /* Chat messages */
    <div className='flex-1 overflow-y-auto p-4 space-y-3'>
      {messages.map((m, i) => (
        <div key={i} className={`flex ${m.role==='user'? 'justify-end': 'justify-start'}`}>
          <div className={`max-w-[75%] p-3 rounded-xl text-sm ${
            m.role==='user'
              ? 'bg-blue-600 text-white'
              : 'bg-gray-100 text-gray-800'
            }`}>{m.text}</div>
        </div>
      ))}
      {loading && <div className='text-gray-400 text-sm'>Thinking...</div>}
    </div>
    /* Input */
    <div className='p-3 border-t flex gap-2'>
      <input
        value={input} onChange={e => setInput(e.target.value)}
        onKeyDown={e => e.key === 'Enter' && sendMessage()}
        placeholder='Ask about your study materials...'
        className='flex-1 border rounded-lg px-3 py-2 text-sm outline-none'
      />
      <button onClick={sendMessage} disabled={loading}
        className='bg-blue-600 text-white px-4 py-2 rounded-lg text-sm'>
        Send
      </button>
    </div>
  </div>
);
}

```


15. End-to-End Flow Diagram

Phase	Stage	Action	File
INGESTION	Teacher	→ Upload Form	NoteUploadForm.tsx
	API	→ POST /api/notes/upload	route.ts
	Storage	→ Cloudinary upload_stream	uploadPDF.ts
	Extract	→ pdf-parse text extraction	extractPDFText.ts
	Chunk	→ chunkText() sliding window	chunkText.ts
	Embed	→ Gemini text-embedding-004	generateEmbedding.ts
	Store	→ pgvector INSERT INTO note_chunks	storeEmbeddings.ts
QUERY	Student	→ Chatbot UI input	RAGChatbot.tsx
	API	→ POST /api/chat	route.ts
	Embed	→ Gemini embed question	generateEmbedding.ts
	Search	→ pgvector cosine similarity	ragQuery.ts
	Prompt	→ context + question → Gemini LLM	ragQuery.ts
	Answer	→ Grounded response displayed	RAGChatbot.tsx

16. Testing Checklist

■ Cloudinary

- ■ Upload a PDF via the form → verify Cloudinary dashboard shows file under pyramidEdu/notes/pdfs/
- ■ Try uploading a .jpg file → expect error: 'Only PDF files allowed'
- ■ Try uploading a PDF > 10MB → expect error: 'File must be 10 MB or less'
- ■ Verify secure_url is returned and accessible in browser

■ Text Extraction

- ■ Upload a text-based PDF → verify text is extracted (check server logs)
- ■ Upload a scanned/image PDF → expect clear error message
- ■ Check extracted text has no excessive whitespace or special characters

■ Chunking & Embeddings

- ■ After upload, query: `SELECT COUNT(*) FROM note_chunks WHERE note_id = "`
- ■ Verify chunk_index is sequential (0, 1, 2, ...)
- ■ Verify embedding column is not NULL: `SELECT id FROM note_chunks WHERE embedding IS NULL`

■ pgvector Search

- ■ Run a similarity search manually in Neon SQL Editor
- ■ Verify top results are semantically related to query
- ■ Test with threshold: confirm chunks with similarity < 0.5 are excluded

■ RAG Chatbot

- ■ Ask a question covered by uploaded notes → verify accurate answer
- ■ Ask a question NOT covered by notes → verify 'Not available in study materials' response
- ■ Ask follow-up questions to test context grounding
- ■ Verify conversation is stored in chat_messages table

17. Troubleshooting & Tips

■ 'vector' type not found in Prisma schema

→ You likely forgot to enable pgvector or add previewFeatures. Run `CREATE EXTENSION IF NOT EXISTS vector;` in Neon SQL Editor, then add `previewFeatures = ["postgresqlExtensions"]` and `extensions = [vector]` to `schema.prisma`.

■ Cloudinary upload fails with 'Invalid image file'

→ Make sure you are using `resource_type: 'raw'` in the `upload_stream` options. PDFs are not 'image' type in Cloudinary.

■ pdf-parse returns empty string

→ The PDF is likely scanned (image-based). pdf-parse only works with digital/text PDFs. Ask the teacher to re-upload a text-based PDF, or add OCR via `tesseract.js`.

■ Gemini API rate limit errors

→ Increase `delayMs` in `generateEmbeddingsBatch()` to 500ms or higher. On free tier, you get 1500 embedding requests/minute.

■ pgvector similarity search returns no results

→ Check: (1) embedding column is not NULL, (2) the IVFFlat index was created, (3) similarity threshold is not too high — try lowering from 0.5 to 0.3.

■ Next.js FormData body size limit

→ Next.js has a 4MB body limit by default. Add to `next.config.js`: `api: { bodyParser: { sizeLimit: '11mb' } }` — or use edge config.

■ TypeScript error: 'Unsupported' field in Prisma

→ This is expected for vector columns. Use `$executeRaw` and `$queryRaw` for all operations on the embedding column.

■ Performance Tip: For large institutions with many uploaded notes, consider filtering the vector search by subject: `WHERE n.subject = $subject AND n.is_active = true` — this reduces the search space and improves both speed and answer accuracy.

■ Security Tip: Always verify the teacher's JWT token in the upload API route before processing. Only users with `role=TEACHER` or `role=ADMIN` should be able to reach `POST /api/notes/upload`.